

APUNTE 2

Autenticación y Autorización: Gestión de Contraseñas

La seguridad en el almacenamiento de credenciales es uno de los pilares fundamentales en cualquier sistema operativo. Si un atacante logra acceder a las contraseñas de los usuarios, tiene acceso a todo el sistema. Por eso, los sistemas modernos no almacenan contraseñas en texto plano, sino que utilizan mecanismos criptográficos para que las contraseñas sean irrecuperables incluso si alguien accede a los archivos del sistema.

En este apunte vamos a recorrer cómo evolucionaron estos mecanismos, qué estructuras usan los sistemas UNIX/Linux para almacenar credenciales, cuáles son los algoritmos de hashing más utilizados, qué es el salt y cómo se aplican las políticas de seguridad en contraseñas.

1. Evolución del Almacenamiento de Contraseñas

Para entender el estado actual de la seguridad en contraseñas, conviene conocer cómo llegamos hasta acá. El recorrido histórico nos muestra qué falló en el pasado y por qué hoy usamos los métodos que usamos.

1.1 Los primeros UNIX: /etc/passwd con DES

En los primeros sistemas UNIX (década de 1970), las contraseñas se almacenaban directamente en el archivo /etc/passwd. Este archivo era (y sigue siendo) legible por cualquier usuario del sistema, ya que contiene información básica de todos los usuarios como el nombre, el UID, el GID, el directorio home y el shell.

Para no guardar la contraseña en texto plano, se la cifraba usando el algoritmo DES (Data Encryption Standard). Una entrada típica se veía así:

Acceso al archivo /etc/passwd

```
sudo cat /etc/passwd
```

```
# Contenido de /etc/passwd en sistemas UNIX antiguos
root:3DgYw1NRgn20Y:0:0:El
Administrador:/root:/bin/bash
alice:xfW1ZqJt4oXU:1001:1001:Alice:/home/alice:/bin/bash

# Formato de cada campo (separados por :)
# usuario : hash_contraseña : UID : GID : descripción : home : shell
```

root	→	Login Name
3DgYwINRgn20Y	→	Password encriptada (actualmente en /etc/shadow)
0	→	UID
0	→	GID
El Administrador	→	Descripción
/root	→	Directorio Home
/bin/bash	→	Shell a ejecutar

El campo 3DgYwINRgn20Y es el hash DES de la contraseña. Este enfoque tenía dos problemas críticos que lo hacían inseguro:

- **Acceso público:** cualquier usuario podía leer /etc/passwd. Eso significa que cualquiera podía obtener los hashes de todos los usuarios y atacarlos de forma offline, sin límite de intentos.
- **Algoritmo débil:** DES utiliza una clave de solo 56 bits y no usaba salts (valores aleatorios para diferenciar hashes). Esto lo hacía vulnerable a ataques de fuerza bruta y a tablas rainbow precomputadas.

En 1998, el proyecto EFF DES Cracker logró descifrar una clave DES en solo 56 horas usando hardware especializado. Esto confirmó que DES ya no era aceptable para uso en producción.

1.2 La solución: /etc/shadow

Para resolver estos problemas, BSD 4.3 (1987) introdujo el archivo `/etc/shadow`. La idea central es simple: separar la información pública del usuario (que sigue en `/etc/passwd`) de los datos sensibles de autenticación (que van al nuevo `/etc/shadow`).

El archivo `/etc/shadow` solo puede ser leído por el usuario `root`, lo que elimina el acceso libre a los hashes. Además, usa algoritmos de hashing modernos con soporte para salts.

vivek:\$1\$fnfffc\$pGteyHdicpGOffXX4ow#5:13064:0:99999:7:::
 ↓ ↓ ↓ ↓ ↓ ↓
 1 2 3 4 5 6

Una entrada típica de `/etc/shadow` tiene el siguiente formato:

```
sudo cat /etc/shadow
```

```
# Formato de /etc/shadow
alice:$y$j9T$rSt7uLbJ$Fgv...Z7o/:19678:0:99999:7:::

# Desglose de cada campo:
alice           → Nombre de usuario
$y$j9T$...     → Hash de la contraseña (incluye algoritmo + salt + hash)
19678          → Días desde el 01/01/1970 (epoch) del último cambio
0              → Días mínimos antes de poder cambiar la contraseña
99999          → Días máximos de validez de la contraseña
7              → Días de advertencia antes de la expiración
(vacío)        → Días de gracia tras vencimiento
(vacío)        → Fecha de expiración de la cuenta
(vacío)        → Reservado
```

1.3 Comparación entre `/etc/passwd` y `/etc/shadow`

El archivo `/etc/passwd`, almacena información básica de usuarios, como UID, GID y shell, junto con hashes de contraseñas. Originalmente, estos hashes se generaban con DES (Data Encryption Standard), un algoritmo obsoleto hoy debido a su clave de 56 bits y falta de *salts*. Por ejemplo, una entrada típica en sistemas antiguos era:

```
root:xfW1ZqJt4oXU:0:0::/root:/bin/bash
```

Aquí, `xfW1ZqJt4oXU` es un hash DES vulnerable a ataques de fuerza bruta, especialmente porque `/etc/passwd` tenía permisos `-rw-r--r--`, permitiendo que cualquier usuario leyera los hashes.

En cambio, `/etc/shadow`, restringe el acceso a `root` mediante permisos `-rw- --- ---` y almacena hashes modernos junto con metadatos de seguridad. Una entrada en `/etc/shadow` incluye:

```
alice:$y$j9T$rSt7uLbJ$Fgv...Z7o/:19678:0:99999:7:::
```

`$2$`: Hash cifrado con algoritmo moderno (e.g., `bcrypt`, identificado por `$y$`).

Por otro lado; el archivo `/etc/shadow` solo es legible por `root`, lo que limita ataques locales. Sin embargo, si un atacante obtiene acceso `root` (mediante exploits como *Dirty Pipe*), puede leer el archivo directamente. Para contrarrestar esto, sistemas como SELinux y AppArmor implementan Mandatory Access Control (MAC), restringiendo incluso a `root` si se configuran en modo *enforcing*.

Herramientas como LUKS (Linux Unified Key Setup) cifran particiones que contienen `/etc/shadow`, protegiendo hashes en caso de robo físico. Por ejemplo, en Ubuntu, el directorio `/etc` suele residir en una partición cifrada con AES-XTS de 256 bits.

Característica	/etc/passwd	/etc/shadow
Permisos	-rw-r--r-- (644)	-rw----- (600)
Acceso	Cualquier usuario	Solo root
Contiene	Info básica del usuario	Hashes y políticas de expiración
Algoritmo hash	DES (obsoleto)	bcrypt, yescrypt, Argon2
Usa salt	No	Sí

Para ver /etc/shadow necesitás privilegios de root: `sudo cat /etc/shadow`. Si ejecutás `cat /etc/shadow` como usuario normal, el sistema te denegará el acceso.

2. Métodos de Hashing de Contraseñas

Un hash criptográfico es una función que toma un dato de entrada (la contraseña) y produce una salida de longitud fija que no puede revertirse para obtener el original. Esto es clave: si alguien obtiene el hash, no puede recuperar la contraseña directamente.

A lo largo del tiempo se fueron desarrollando distintos algoritmos. Veamos los más importantes:

2.1 DES — Obsoleto

Fue el primero en usarse en UNIX. Como vimos, tiene claves de 56 bits y no usa salt. Hoy es completamente inseguro y no debe usarse. Solo se menciona por razones históricas.

2.2 MD5 (\$1\$) y SHA-1 (\$sha1\$)

Fueron adoptados como mejora sobre DES y ya incorporaron el uso de salts aleatorios. Sin embargo, su principal problema es la velocidad: están diseñados para ser rápidos, lo cual es malo para el almacenamiento de contraseñas.

¿Por qué la velocidad es un problema? Porque un atacante con una GPU moderna puede probar cientos de millones de hashes MD5 por segundo. Si el algoritmo es rápido, el ataque de fuerza bruta también lo es.

```
# Ejemplo de hash MD5 en /etc/shadow
$1$rSt7uLbJ$Fgv...Z7o/

# Identificación del formato:
# $1$ → indica algoritmo MD5
# rSt7uLbJ → salt aleatorio
# Fgv...Z7o/ → hash resultante
```

2.3 SHA-512 (\$6\$)

SHA-512 es parte de la familia SHA-2. Produce hashes de 512 bits y es significativamente más seguro que MD5. Durante mucho tiempo fue el estándar en distribuciones como Ubuntu y Debian. Sigue siendo aceptable, aunque existen alternativas mejores.

```
# Ejemplo de hash SHA-512 en /etc/shadow
user:$6$aei123$abc456def789ghijklmno...:19678:0:99999:7:::

# $6$ → SHA-512
# aei123 → salt
# abc456... → hash
```

2.4 bcrypt (\$2b\$ / \$2y\$)

Desarrollado en 1999 por Niels Provos y David Mazieres, bcrypt fue el primer algoritmo diseñado específicamente para almacenamiento de contraseñas. Sus características principales son:

- **Costo ajustable:** permite configurar cuántas iteraciones realiza. A mayor costo, más tiempo tarda cada cálculo, lo que ralentiza los ataques de fuerza bruta.
- **Salt incorporado:** usa un salt de 128 bits generado automáticamente.
- **Resistencia a hardware especializado:** requiere acceso rápido a memoria, lo que dificulta su aceleración con ASICs o FPGAs.

```
# Ejemplo de hash bcrypt en /etc/shadow
$2y$12$rSt7uLbJFgvZ7o/xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

# $2y$ → algoritmo bcrypt
# 12 → factor de costo (2^12 = 4096 iteraciones)
# Los siguientes 22 caracteres → salt
# El resto → hash
```

Error común: el identificador \$1\$ corresponde a MD5, NO a bcrypt. bcrypt usa \$2b\$ o \$2y\$. Este es un error frecuente que hay que tener claro.

2.5 PBKDF2

PBKDF2 (Password-Based Key Derivation Function 2) aplica una función como HMAC-SHA256 miles de veces de forma repetida. El número de iteraciones es configurable y se almacena en el hash. Se usa en FreeBSD y en muchas aplicaciones web.

```
# Ejemplo de hash PBKDF2 en FreeBSD
$pbkdf2-sha256$29000$rSt7uLbJ$Fgv...Z7o/

# 29000 → número de iteraciones
# rSt7uLbJ → salt
# Fgv...Z7o/ → hash
```

Su principal debilidad es que SHA-256 puede acelerarse con GPUs si el número de iteraciones es bajo. Se recomienda usar al menos 100.000 iteraciones en implementaciones modernas.

2.6 Argon2 — El estándar actual

Argon2 ganó el Password Hashing Competition en 2015 y es el algoritmo recomendado actualmente por el NIST. Existen tres variantes: Argon2d, Argon2i y Argon2id. Esta última combina las ventajas de las dos anteriores y es la más recomendada. Sus parámetros configurables son:

- **m (memoria):** cantidad de RAM que consume el cálculo. Esto lo hace resistente a aceleración con GPUs, que tienen memoria limitada por núcleo.
- **t (tiempo/iteraciones):** número de pasadas sobre la memoria.
- **p (paralelismo):** cantidad de hilos de ejecución.

```
# Ejemplo de hash Argon2id en /etc/shadow
$argon2id$v=19$m=65536,t=3,p=2$rSt7uLbJ$Fgv...Z7o/

# argon2id → variante del algoritmo
# v=19      → versión del algoritmo
# m=65536   → usa 64 MiB de memoria
# t=3       → 3 iteraciones
# p=2       → 2 hilos paralelos
# rSt7uLbJ  → salt
# Fgv...Z7o/ → hash
```

Argon2 requiere aproximadamente 1000 veces más recursos que bcrypt para ataques masivos, gracias al uso intensivo de memoria.

2.7 yescrypt — El predeterminado actual en Debian/Ubuntu

Desde Debian 11 (Bullseye, 2020) y Ubuntu 22.04 (Jammy Jellyfish, 2022), el algoritmo predeterminado es yescrypt. Es una evolución de scrypt, diseñada para ser aún más resistente a hardware especializado (ASICs y FPGAs). Kali Linux, al ser derivado de Debian, también lo usa por defecto.

2.8 Tabla comparativa de algoritmos

Algoritmo	Año	Salt	Velocidad	Resistencia GPU	Estado
DES	1970s	No	Muy rápido	Muy baja	Obsoleto
MD5 (\$1\$)	1990s	Sí	Rápido	Baja	Inseguro
SHA-512 (\$6\$)	2000s	Sí	Moderado	Media	Aceptable
bcrypt (\$2b\$)	1999	Sí	Lento	Alta	Recomendado
PBKDF2	2000	Sí	Configurable	Media-Alta	Aceptable
Argon2id	2015	Sí	Configurable	Muy alta	Estándar NIST
yescrypt	2020	Sí	Configurable	Muy alta	Default Debian/Ubuntu

3. El Salt: Por Qué Es Esencial

El salt es un valor aleatorio que se genera para cada usuario y se combina con su contraseña antes de aplicar el hash. Aunque el salt se almacena junto al hash (en texto visible en `/etc/shadow`), su función no es mantenerse en secreto sino garantizar que los hashes sean únicos.

3.1 El problema sin salt

Imaginemos que dos usuarios tienen la misma contraseña (por ejemplo, "123456"). Si el sistema no usa salt, ambos van a tener exactamente el mismo hash almacenado. Un atacante que vea los hashes va a notar la coincidencia y va a saber que esos dos usuarios comparten contraseña. Peor aún, si descifra la contraseña de uno, automáticamente descifra la del otro.

```
# Sin salt: misma contraseña → mismo hash (INSEGURO)
alice: $hash_de_123456
bob:   $hash_de_123456 ← el atacante ve que son iguales
```

3.2 El problema de las Rainbow Tables

Una rainbow table es una base de datos precalculada que contiene miles de contraseñas comunes junto con su hash. Si el sistema no usa salt, un atacante puede tomar cualquier hash del sistema y buscarlo directamente en la tabla para obtener la contraseña original en cuestión de segundos.

```
# Rainbow table simplificada (sin salt el ataque es trivial):
password123 → 5f4dcc3b5aa765d61d8327deb882cf99
123456      → e10adc3949ba59abbe56e057f20f883e
admin       → 21232f297a57a5a743894a0e4a801fc3
qwerty      → d8578edf8458ce06fbc5bb76a58c5ca4
```

3.3 La solución: el salt

Con salt, aunque dos usuarios tengan la misma contraseña, sus hashes son completamente distintos porque cada uno tiene su propio salt aleatorio:

```
# Con salt: misma contraseña → hashes distintos (SEGURO)
alice: $6$xK9mP2qL$hash_de_(123456+xK9mP2qL)
bob:   $6$rTv8nW3j$hash_de_(123456+rTv8nW3j) ← hashes completamente distintos

# Las rainbow tables precomputadas ya no sirven porque
# habría que calcular una tabla diferente para cada salt posible.
```

3.4 Características de un buen salt

- **Aleatorio:** debe generarse con un generador criptográficamente seguro. Un salt predecible puede ser explotado.

- **Único por usuario:** cada usuario debe tener su propio salt. Nunca reutilizar el mismo salt para varios usuarios.
- **Longitud mínima:** se recomienda al menos 16 bytes (128 bits).
- **Almacenado junto al hash:** el salt se guarda en texto plano en /etc/shadow. Esto es correcto: su función no es ser secreto sino ser único.

3.5 Generación de salt y hash en la práctica

```
# Paso 1: Generar un salt aleatorio de 16 bytes en base64
openssl rand -base64 16
# Salida ejemplo: K3mX9pQrLvTy8wZn==

# Paso 2: Generar un hash SHA-512 combinando contraseña y salt
echo -n 'micontraseña_segura' | sha512sum

# Paso 3: En la práctica, el sistema lo hace automáticamente
# al usar comandos como passwd o al crear usuarios con adduser.

# También podés generar un hash compatible con /etc/shadow así:
python3 -c "import crypt; print(crypt.crypt('micontraseña',
crypt.mksalt(crypt.METHOD_SHA512)))"
```

Referencias Bibliográficas

- Silberschatz, A. — Operating System Concepts, Cap. 14 (Protection)
- APUE (Advanced Programming in the UNIX Environment), Cap. 6
- NIST SP 800-63B — Digital Identity Guidelines
- Provos, N. & Mazières, D. (1999) — A Future-Adaptable Password Scheme (bcrypt)
- OSTEP, Cap. 39 (Security)